

Documentação do Projeto: Zhivago HotelApp

1. O Mini-Mundo (Contexto de Negócio)

O **Zhivago HotelApp** é uma plataforma de marketplace focada em aluguéis por temporada e venda de imóveis, seguindo um modelo de negócio semelhante ao Airbnb. No ecossistema da aplicação, existem duas personas principais: o **Anfitrião (Proprietário)** e o **Hóspede (Cliente)**.

O anfitrião pode cadastrar seus imóveis na plataforma, informando dados como título, descrição, preço, quantidade de quartos e vagas, além de fotos. Os hóspedes podem navegar por um feed de imóveis disponíveis e visualizar os detalhes. Ao escolher um imóvel para aluguel, o hóspede deve selecionar as datas no calendário e enviar uma solicitação de reserva.

A plataforma não aprova a reserva automaticamente. A solicitação entra no status *Pendente* e o anfitrião é notificado para *Aprovar* ou *Recusar* a estadia. Todas as negociações entre o hóspede e o anfitrião ocorrem por meio de um **Chat em Tempo Real**, onde notificações automáticas (Push) e mensagens do sistema mantêm ambas as partes atualizadas sobre o status da reserva. Após uma reserva concluída, o hóspede pode avaliar o imóvel, gerando uma nota e feedback público.

Tanto anfitriões quanto hóspedes possuem autonomia: o hóspede tem a aba "Minhas Viagens" para acompanhar e cancelar reservas se necessário, e o anfitrião possui o "Meu Dashboard" com gráficos financeiros de faturamento e uma lista gerencial de reservas recebidas.

2. Arquitetura e Stack Tecnológica

A aplicação foi dividida em dois grandes blocos, unificando as melhores tecnologias modernas para escalabilidade e fluidez:

- Frontend (Mobile & Web):** Desenvolvido utilizando **React Native** com o framework **Expo** (Expo Router para navegação em abas e stack). Estilização via StyleSheet e ícones vetoriais.
- Backend (API Rest):** Construído com **Node.js** utilizando o micro-framework **Fastify**, garantindo alta velocidade de requisição.
- Banco de Dados:** Utilização do **SQLite** (banco relacional leve) intermediado pelo ORM **Prisma**, que garantiu tipagem rigorosa e facilidade de migrações.
- Tempo Real & Notificações:** Utilização de **Socket.io** para comunicação bidirecional no chat e serviços de push notification nativos do Expo.
- Uploads de Imagens:** Processamento e redimensionamento automático de imagens utilizando a biblioteca **Sharp** (conversão para WebP e limitação de largura).

3. Principais Funcionalidades Desenvolvidas

Desde o início do projeto até a fase final de lapidação, entregamos as seguintes soluções:

- Autenticação Segura:** Login, Registro e recuperação com JWT (JSON Web Tokens).
- CRUD Completo de Imóveis:** Criação, listagem, edição e exclusão de anúncios, com suporte a upload de fotos.
- Fluxo de Reservas (Booking):** Sistema de reservas com validação de datas no calendário, impedindo *overbooking* (dupla locação).
- Comunicação em Tempo Real (Chat):** Conversas em tempo real isoladas por transação, garantindo segurança na negociação.
- Dashboard Financeiro (Anfitrião):** Gráficos mensais de receita e gestão de reservas ativas usando *react-native-chart-kit*.
- Gerenciamento de Viagens (Hóspede):** Visualização do histórico com possibilidade de cancelamento autônomo e avaliação.
- Integração Externa:** Redirecionamento protegido e avisos de segurança ao optar pelo contato direto via WhatsApp.

4. Desafios Enfrentados e Erros Superados

O desenvolvimento envolveu arquiteturas complexas que trouxeram desafios técnicos interessantes:

- Desafio do IP Dinâmico:** Como o Expo roda em LAN e o servidor local muda de IP a cada rede Wi-Fi, enfrentamos vários erros de *Network Error*. **Solução:** Criamos scripts em Node (*update-ip.js*) que identificam a interface de rede da máquina e injetam automaticamente o IP correto no arquivo *.env* e no código, evitando configurações manuais diárias.
- Escopo e Referências no Servidor (ReferenceError):** Durante a unificação das rotas de reserva e chat, tivemos um bug onde a reserva não criava porque a variável *conversation* estava dentro de um bloco condicional (*if*) e não podia ser acessada pelo fluxo final. **Solução:** O escopo foi corrigido movendo a declaração para fora e adicionando tratamento robusto de erros (*catch*).
- Estados React na Paginação:** Ao adicionar paginação (5 itens) na lista do Dashboard do anfitrião para economizar memória, a variável de estado global *loading* foi sobrescrita, gerando o erro de *Identifier already been declared*. **Solução:** Refatoramos a arquitetura do Dashboard, dividindo em dois carregamentos distintos (Estatísticas e Reservas) e nomeando claramente as variáveis (*bookingsChartData* vs *bookingsData*).
- Limpeza de Código e Performance:** O projeto acumulou arquivos inutilizados na transição de telas estáticas para dados da API. Utilizamos ferramentas de análise estática (*knip* e *tsc*) para varrer e remover dependências e exportações não utilizadas de forma cirúrgica.
- Testes de Estresse (K6):** Precisávamos validar a resistência da API Fastify. Enfrentamos erros do terminal Windows para rodar o K6, que foram solucionados com scripts paralelos, garantindo que o servidor aguentasse requisições infinitas simuladas.

5. Modelagem do Banco de Dados (Prisma Schema)

A estrutura relacional das tabelas ficou definida como:

- User:** *id, name, email, password, role* (ADMIN/USER).
- Listing (Imóvel):** *id, name, price, type, location*, relacionando-se diretamente ao *ownerId* (User).
- Booking (Reserva):** Relacionamento de N:N gerido entre User e Listing, controlando *startDate, endDate* e *status*.
- Review (Avaliação):** Contém *rating* e *comment*, ligando o Hóspede ao Imóvel.
- Conversation & Message:** Estrutura de Chat onde *Conversation* liga 2 participantes a um Imóvel, e *Message* armazena o texto e o emissor.